GUJARAT TECHNOLOGICAL UNIVERSITY

Program Name: Diploma in Engineering

Level: Diploma

Branch: Computer Engineering / Computer Science and Engineering

Subject Code : DI04000021

Subject Name : Object Oriented Programming-Java

W.E. F. Academic Year:	2025-26
Semester:	4th
Category of the Course:	PCC

Prerequisite:	The recommended pre-requisite language is C (or any other structured programming language like Python, C++)
Rationale:	Object-Oriented Programming (OOP) has become the standard paradigm for developing modern software applications due to its ability to model real-world problems effectively. Introducing students to OOP using Java equips them with essential programming skills that are highly relevant in both academia and industry. Java is one of the most widely used programming languages, known for its platform independence (Write Once, Run Anywhere), security, and rich library support, making it a perfect language to teach the principles of OOP. By including OOP with Java in the curriculum, students will be able to understand the core OOP concepts, develop problem solving skills and industry relevant skills as Java is extensively used in enterprise applications, Android development, web development, and emerging fields like AI and Machine Learning.

Course Outcome:

After Completion of the Course, Student will able to:

No	Course Outcomes	RBT Level
01	Develop and execute basic Java programs using data types, control structures, and arrays.	Apply
02	Apply object-oriented principles by creating classes and objects to model real-world concepts.	Apply
03	Implement modular, reusable and extensible programs using inheritance, interfaces and abstraction.	Apply
04	Implement exception handling and collection framework to manage runtime errors and data effectively.	Apply
05	Demonstrate modern Java features.	Apply

**Revised Bloom's Taxonomy (RBT)*

Teaching and Examination Scheme:

Teaching Scheme (in Hours)			Total Credits L+T+ (PR/2)	Assessment Pattern and Marks				Total Marks
				Theory		Tutorial / Practical		
L	T	PR	C	ESE (E)	PA (M)	PA (I)	ESE (V)	
3	0	2	4	70	30	20	30	150

Course Content:

Unit No.	Content	No. of Hours	% of Weightage
1.	**Introduction to Java Programming** **Introduction to Java** • Introduction to Java • Java Features • Java Applications **Writing & Running First Program** • JDK, JRE, JVM • Byte Code • Structure of a Java program (class, main()). • Hello World program – compile & run. **Data Types & Variables** • Primitive types (byte, short, int, long, float, double, char, boolean). • Declaring & initializing variables • The Scope and Lifetime of Variables • Type Conversion and Type Casting • Brief overview of Operators (Arithmetic, Bitwise, Rational, Logical, Assignment, Conditional, Ternary, Increment and Decrement). **Control Flow** • if-else (simple & nested & ladder). • switch-case	10	20

	• for loop, while loop, do-while • Jump statements: break, continue **Arrays** • 1D arrays: declaration, initialization, traversal. • 2D arrays: introduction Problem Solving practice on the above concepts		
2.	**Object-Oriented Programming Concepts** **Principles of OOP** • Abstraction • Encapsulation • Inheritance • Polymorphism **Classes & Objects** • Class • Object • Fields and Methods **Constructors** • Default Constructor • Parameterized Constructor • Constructor Overloading **Important Keywords** • this • static-static class, static block, static method, static variable **Access Modifiers** • public • private • protected • default **Methods** • Method Definition • Method Calling	10	25

	• Method Overloading **Strings** • String Class • Common String Methods (length(), charAt(), substring(), toUpperCase(), toLowerCase()) **Wrapper Classes** • Wrapper Classes (Byte, Short, Integer, Long, Float, Double, Character, Boolean) • Autoboxing • Unboxing **User Input** • Scanner Class • Command Line Arguments **Garbage Collection** • finalize() Method Problem Solving practice on the above concepts		
3.	**Inheritance, Interfaces & Abstraction** **Inheritance** • Concept of Inheritance • Need of Inheritance • Types of Inheritance • super keyword • Method Overriding • Dynamic Method Dispatch • Final Keyword **Abstract Classes** • Abstract Class • Abstract Methods • Rules of Abstract Class **Interfaces** • Need of interface • Interface Declaration	10	25

	• Interface Implementation • Multiple Inheritance using Interface • Default and Static Methods in Interface **Packages** • Defining a Package • Importing a Package • Built-in Packages (java.util, java.lang, etc.) Problem Solving practice on the above concepts		
4.	**Exception Handling & Collections Framework** **Errors and Exceptions** • Types of Errors (Compile-time, Runtime, Logical) • Exception Hierarchy (Throwable, Exception, RuntimeException) **Exception Handling** • try block • catch block • finally block • throw keyword • throws keyword • Multiple catch blocks • Built-in Exceptions (ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException, etc.) **Collections Framework** • Need of Collections • Collection Hierarchy Overview • List Interface ○ ArrayList: introduction, add/remove/search elements • Set Interface ○ HashSet :introduction, unique elements • for-each loop	8	20

	Problem Solving practice on the above concepts		
5.	**Modern Java Features** **Basic Overview of Java Language Version Updates** • Recent Java Features and Basic Overview of major Java enhancements from Java 7 onwards (e.g., try-with-resources, lambda expressions, modules, records, pattern matching). **Functional Features (Java 8 Intro)** • Lambda Expression – syntax and simple example (e.g. print list elements). • Stream API – introduction with filter, map, forEach operations on a list. **Introduction to AI with Java** • Meaning of AI and Machine Learning: basic definitions • Role of Java in AI (used in enterprise apps, data handling, API integration) • Basic overview of AI libraries in Java: o Deeplearning4j (DL4J) – deep learning in Java o TensorFlow Java API – running pre-trained models	7	10
	Total	**45**	**100**

Suggested Specification Table with Marks (Theory):

Distribution of Theory Marks (in %)					
R Level	**U Level**	**A Level**	**N Level**	**E Level**	**C Level**
26	**44**	**30**	**0**	**0**	**0**

Where R: Remember; U: Understanding; A: Application, N: Analyze and E: Evaluate C: Create (as per Revised Bloom's Taxonomy)

References/Suggested Learning Resources:

(a) Books:

1. Horstmann, Cay S. and Gary Cornell, "Core Java Volume I – Fundamentals", Prentice Hall, Latest Edition
2. Schildt, Herbert, "Java: The Complete Reference", McGraw-Hill Education, Latest Edition
3. Sierra, Kathy and Bert Bates, "Head First Java", O'Reilly Media, Latest Edition

4. Balagurusamy, E., "Programming in Java", McGraw-Hill Education, New Delhi, Latest Edition
5. Krishna, P. Radha, "Java Programming", CBS Publishers & Distributors, New Delhi, Latest Edition
6. Artificial Intelligence with Java and Deeplearning4j: Master AI Development with Java's Most Powerful Deep Learning Framework -by THOMPSON CARTER

(b) Open source software and website:

1. Eclipse IDE – https://www.eclipse.org/ide/
2. OpenJDK (Java Development Kit) – https://openjdk.org/
3. Java Virtual Labs (IIT Delhi) for learning and practice – https://java-iitd.vlabs.ac.in/
4. Java Programming Tutorial by TutorialsPoint – https://www.tutorialspoint.com/java/index.htm
5. Java Tutorial for Beginners by JavaTpoint – https://www.javatpoint.com/java-tutorial
6. Official Oracle Java Tutorials – https://docs.oracle.com/javase/tutorial/

Suggested Course Practical List:

1. Create simple programs in Java.
   a. Create a Java program to print "Hello World", then compile, debug and execute it using Java compiler and interpreter.
   b. Create a Java program to perform arithmetic operations and demonstrate type casting.
2. Create Java programs using control structures.
   a. Create a Java program to find maximum of three numbers.
   b. Create a Java program to find sum of digits of a number.
3. Create Java programs using arrays.
   a. Create a Java program to find sum and average of array elements.
   b. Create a Java program to perform addition of two matrices.
4. Create a Student class with fields and methods in Java program. Demonstrate object creation and method calling.
5. Create Java programs using constructor, this and static keywords.
   a. Create a Java program to demonstrate default and parameterized constructors and the use of 'this' keyword.
   b. Create a Java program to demonstrate different uses of 'static' keyword.
6. Create a Java program to demonstrate method overloading.
7. Create a Java program to perform various String operations of using built-in String methods.
8. Create a Java program to accept input using the Scanner class, store values and demonstrate autoboxing and unboxing.

9. Create a Java program to demonstrate single and multilevel inheritance. (e.g., class Person → Employee → Manager)
10. Create an abstract class Shape having abstract method area() for calculating the area of a shape and concrete method displayType() to display the type of the shape. Define an interface Drawable with a method draw() that specifies how the shape should be displayed. Implement class Circle/Rectangle that extend a Shape and implement Drawable, demonstrating method overriding and multiple inheritance.
11. Write a Java program to demonstrate the use of the 'final' keyword.
12. Create a Java program to create a user-defined package containing at least one class. Import this package in another program and demonstrate the usage of the class and its methods.
13. Create a Java program to demonstrate exception handling using built-in exceptions (divide by zero, null pointer). Show how exceptions are caught and handled using try-catch-finally.
14. Write a Java program to demonstrate the use of Java Collections Framework. Use ArrayList and HashSet to perform operations like add, remove, search, and traversal. Display the elements of each collection using a for-each loop.
15. Write a Java program to demonstrate the use of Lambda Expression and Stream API:
    - To filter even numbers
    - To convert strings to uppercase

List of Laboratory/Learning Resources Required:

1. Computer with basic configuration with windows or unix os
2. Text Editor (Notepad++, Visual Studio Code, Eclipse IDE, NetBeans IDE or any other editor)
3. Software: JDK (Java Development Kit) Version 8 or above

Suggested Activities for Students:

1. Case Study on OOP Concepts in Real Applications – analyze how real-world systems like a hospital management system or e-commerce cart use inheritance, interfaces, and abstraction.
2. Case Study on Exception Handling in Critical Systems – explore how banking or railway reservation systems handle runtime errors (invalid inputs, failed transactions) using exceptions for data safety.
3. Case Study on Collections Framework Usage – study how ArrayList and HashSet are applied in social media apps (friend lists, posts feed, and unique hashtags).
4. Mini Project: Student Information Manager – implement CRUD operations using OOP concepts, file handling, and exception handling (like mini-database using text files).

5. Mini Project: Quiz/Test System – develop a single-user quiz app using OOP, arrays, file handling (for storing questions and score records), and exception handling for input validation.
6. Mini Project: Simple To-Do List with Collections – use ArrayList to create a console-based to-do tracker with functionality to add, edit, remove, and search tasks.
7. Poster: Evolution of Java (Java 7 to Java 16+) – show visually how Java evolved with each version highlighting new features like lambda, streams, records, and sealed classes.
8. Poster: Exception Hierarchy – prepare a diagrammatic poster showing the hierarchy of Throwable, Exception and RuntimeException.
9. Poster: OOP Principles in Real World – prepare a visual poster illustrating Abstraction = ATM interface, Encapsulation = Student data fields + getters/setters, Inheritance = Vehicle classes, Polymorphism = method overriding in shapes.
10. Website Exploration: Java Virtual Labs (IIT Delhi) – use https://java-iitd.vlabs.ac.in/ for writing and simulating programs on loops, arrays, and OOP. Prepare a short reflective report.
11. Website Exploration: Oracle Java Documentation – explore Java official documentation focusing on Collections, Streams, and new Java language updates. Write summaries of 2–3 features.
12. Website Exploration: Deeplearning4j and TensorFlow Java API – explore the basic examples and document what kind of AI tasks can be integrated into Java applications.

****